Transcripción de entrevista con [Desarrollador] Entrevistador: Emilio Gómez Montuy. Perfil considerado: programador remoto en un área relacionada con Google Maps.

Emilio: Hola, [Desarrollador], ¿cómo estás? Pues primero que nada, muchas gracias por darte el tiempo. Como te había comentado, estamos trabajando en un proyecto de la universidad con la ADIP que se llama Carto-Visor. Estamos haciendo entrevistas para entender cómo se usa la información de los mapas y qué dificultades hay para consultarla o trabajar con ella.

[Desarrollador]: Hola, Emilio. Bien, gracias. Sí, claro, con mucho gusto. Me contaste un poquito lo del proyecto. Creo que te puedo ayudar más desde la parte de programación y de cómo haces que una herramienta con mapas funcione para quien la va a usar.

Emilio: Sí, justamente eso nos interesa. Ya hablamos con personas que usan información territorial y con alguien que hace tareas administrativas. Ahora queremos entender esa parte de quien desarrolla una herramienta. Te voy a preguntar de tu experiencia en general, sin necesitar nombres de proyectos ni información interna de tu trabajo.

[Desarrollador]: Sí, está bien. Podemos hablar de ejemplos generales. Hay detalles del trabajo que no podría platicar, pero sí de cómo entiendo una solicitud, cómo reviso un problema y qué preguntas haría si estuviera desarrollando algo como lo que me cuentas.

Emilio: Perfecto. Para empezar, ¿nos podrías contar un poquito a qué te dedicas y cómo se relaciona tu trabajo con Google Maps?

[Desarrollador]: Trabajo como programador, en un área relacionada con plataformas de mapas digitales, y trabajo de forma remota. Mi parte es desarrollar y revisar software que trabaja con mapas e información de ubicaciones. Entonces me toca pensar en cómo se muestra la información, qué pasa cuando alguien busca o selecciona algo y cómo detectar errores. Eso no significa que yo decida qué información geográfica es oficial.

Emilio: Y en un día normal, ¿qué te llega para trabajar? O sea, ¿te dicen qué programar y ya, o también te toca entender para qué lo necesitan?

[Desarrollador]: Te puede llegar una función que quieren agregar o un problema que alguien encontró. Pero casi siempre necesitas preguntar algo. Si te dicen “la búsqueda está mal”, tienes que entender qué buscó la persona, qué esperaba y qué apareció. Después revisas el comportamiento, haces el cambio y compruebas que no hayas afectado otra cosa. Programar es una parte; entender bien lo que te están pidiendo te ocupa bastante tiempo.

Emilio: Va. ¿Me podrías contar algún ejemplo de esos problemas, para verlo con algo más concreto? Puede ser uno sencillo y sin entrar en detalles del proyecto.

[Desarrollador]: Sí, a ver, te pongo un ejemplo general de una consulta por nombre. El usuario veía varios resultados, seleccionaba uno y el mapa seguía mostrando la selección anterior. Entonces decía que la ubicasión estaba mal. A primera vista podía parecer que había una coordenada equivocada, pero el problema estaba en cómo se actualizaba la selección en la pantalla. El dato era uno y lo que estaba viendo la persona era otra cosa.

Emilio: Ah, ya. Entonces no necesariamente había que corregir el lugar en el mapa. ¿Cómo te darías cuenta de dónde estaba el problema?

[Desarrollador]: Primero intentaría repetir lo que hizo la persona. Buscar el mismo nombre, elegir el mismo resultado y revisar qué queda seleccionado. Luego compararía lo que recibe el programa con lo que 1

muestra. Si la información que llega corresponde al lugar correcto, ya tienes una pista. También revisaría si sucede siempre o sólo cuando cambias rápido entre resultados. Necesitas ir separando las posibilidades, porque con una captura sola puedes equivocarte de diagnóstico.

Emilio: Y si tú no puedes repetirlo, ¿qué haces? Porque me imagino que a veces a la persona sí le pasa y a ti no.

[Desarrollador]: Sí, pasa. Ahí pediría más contexto: qué hizo antes, qué texto escribió y qué estaba abierto. A veces la persona recuerda el último clic, pero el error depende de algo que hizo dos pasos antes. También puede cambiar según la conexión o el tamaño de la pantalla. No es decirle que está inventando el problema; es conseguir suficiente información para encontrarlo. Si todavía no lo reproduzco, lo dejaría registrado como pendiente de revisar.

Emilio: ¿Qué sería una buena forma de reportarte eso? Algo que sí te sirva para trabajar y no estar preguntando muchas veces.

[Desarrollador]: Que me diga qué quería hacer, los pasos que siguió, qué esperaba ver y qué vio realmente. Una captura ayuda si se entiende qué está señalado, pero no reemplaza los pasos. Si se puede compartir, también sirve un ejemplo de la búsqueda o del archivo que estaba usando. Y decir si pasa siempre. Con eso ya puedo empezar. Un mensaje de “no sirve el mapa” me deja demasiadas cosas por adivinar.

Emilio: En la entrevista con [Analista territorial] salía mucho que puedes tener un mapa que se vea bien, pero que la información no sea la adecuada. Desde tu parte, ¿cómo separas esos problemas?

[Desarrollador]: Yo revisaría qué está fallando en cada caso. Puede haber un dato incorrecto, una interpretación equivocada de ese dato o un error del programa al mostrarlo. Si una división territorial cambió y el archivo sigue siendo anterior, arreglar un botón no lo resuelve. Si el archivo está bien y el programa mezcla dos campos, ahí sí toca corregir el software. Necesitas hablar con quien conoce la información, porque yo no puedo decidir la realidad del territorio viendo el código.

Emilio: Para nuestro proyecto nos explicaron que vamos a empezar con dos capas, sectores y cuadrantes de Seguridad Ciudadana. También nos dijeron que los nombres y las claves ya vienen de esa institución. ¿Qué cuidado tendrías ahí?

[Desarrollador]: Conservaría esas claves tal como están definidas. A veces alguien quiere “limpiar” un campo y termina cambiando algo que sirve para identificarlo. Por ejemplo, si una clave tiene ceros al principio, habría que confirmar que no se pierdan al cargarla. También revisaría qué campo es la clave y cuál es un nombre para mostrar. Si necesitan un texto más fácil para el usuario, pueden explicarlo en la interfaz sin reemplazar el identificador de origen.

Emilio: Va. Ahora quiero hacer el ejercicio de los actores que vimos en clase. Si pensamos en una herramienta de este tipo, ¿qué personas tendrían que participar para que funcione?

[Desarrollador]: Estaría quien entrega y conoce los datos, la persona que define para qué se va a usar la herramienta, quienes la desarrollan y quienes hacen las consultas. Puede haber alguien que revise el diseño o que ayude a probarla. También necesitas ubicar quién puede autorizar un cambio en la información. No todo lo tiene que hacer una sola persona, pero sí debe quedar claro a quién preguntas cuando hay una duda.

Emilio: Y tú, como programador, ¿en qué parte de ese mapa de actores te pondrías? Porque desde fuera a veces pensamos que el programador puede arreglar todo.

[Desarrollador]: Me pondría en la parte de convertir la necesidad en un comportamiento que se pueda probar. Puedo decirte si una función está respondiendo, si un archivo carga o si el programa está mostrando el 2

campo que corresponde. Pero para saber si un cuadrante está bien delimitado necesito a quien conoce esa división. También puedo señalar una inconsistencia, aunque no me toque resolverla por mi cuenta. Esa separación evita que una decisión de programación termine cambiando información oficial.

Emilio: Y cuando trabajas remoto, ¿cómo cambia esa comunicación? ¿Hay algo que se vuelva más difícil por no estar con los demás en la oficina?

[Desarrollador]: Lo que más se nota es cuando falta contexto. En persona puedes hacer una pregunta rápida y alguien te señala la pantalla. Remoto, si me mandan un mensaje y luego no coincidimos en horario, puedo quedarme esperando una aclaración. Por eso intento dejar por escrito qué entendí y qué falta. No necesitas un documento enorme, pero sí algo que otra persona pueda retomar sin haber estado en todas las llamadas.

Emilio: ¿Te ha pasado que dos personas trabajen pensando cosas distintas y se den cuenta hasta después?

[Desarrollador]: Sí. Puede pasar con algo tan sencillo como “mostrar los resultados”. Una persona entiende que se deben ver todos y otra que sólo se muestran los que cumplen un filtro. Si nadie lo aclara, los dos pueden hacer bien su parte y aun así entregar algo que no coincide. También pasa cuando hay dos archivos con el mismo nombre. Ahí una referencia compartida y una descripción corta de lo que esperan ayudan mucho.

Emilio: Ahora, para el mapa del proceso, imagina que nos llega la solicitud de agregar una capa. ¿Cómo ordenarías los pasos desde que la recibes hasta que alguien la consulta?

[Desarrollador]: Primero aclararía qué pregunta va a responder y quién la va a usar. Después revisaría el archivo y la información que lo acompaña. Haría una carga de prueba para ver qué contiene y cómo se muestra. Luego trabajaría la consulta y comprobaría resultados con ejemplos que alguien del área pueda confirmar. Al final probaría el recorrido completo con un usuario. Entre esos pasos puede haber regresos, sobre todo si falta un campo o no está claro qué significa.

Emilio: ¿En qué parte crees que se atora más ese proceso? ¿En programar o en conseguir la información?

[Desarrollador]: Depende, pero muchas veces en las definiciones. Puedes tener el archivo y no saber si es la versión que debes usar, o qué campo necesitas mostrar. También se atora cuando esperas que alguien confirme una interpretación. El código puede avanzar, pero si programas sobre una suposición que luego cambia, tienes que rehacerlo. Yo intentaría encontrar esas dudas temprano y dejar visible cuáles siguen abiertas.

Emilio: En nuestro caso [Directora ADIP] nos comentó que esas dos capas iniciales ya son compatibles. Para este semestre usaríamos las que nos compartan. ¿Qué harías primero con eso?

[Desarrollador]: Haría una prueba sencilla de cargarlas y consultarlas, respetando lo que les entregaron. Si ya les confirmaron que son compatibles, no empezaría inventando un problema de homologación. Revisaría que el programa conserve los campos y que se pueda identificar cada capa. Luego probaría una consulta completa, desde elegir un sector o cuadrante hasta ver su información. Esa prueba les ayuda a descubrir lo que realmente falta en la herramienta.

Emilio: ¿Y si después llegan otras capas? Porque sí nos interesa que más adelante se pueda ampliar.

[Desarrollador]: Ahí dejaría documentado qué información espera el programa de cada archivo. Por ejemplo, qué campo lo identifica y cómo vienen las coordenadas. Si una capa nueva usa otra referencia para ubicar los datos, hay que revisarlo antes de combinarla. Pero eso sería una necesidad que se confirma con esa capa, no algo que ustedes deban asumir de las dos primeras. Ampliar también implica tener a quién preguntarle por la información nueva. 3

Emilio: Cuando dices cómo vienen las coordenadas, ¿te refieres a por qué a veces un punto o una zona se ve en un lugar que no corresponde?

[Desarrollador]: Puede ser una de las razones. Dos archivos pueden expresar la ubicación de maneras distintas y el programa tiene que interpretarlas correctamente. Pero no todo desplazamiento tiene esa causa. También podrías estar viendo un resultado anterior o haber confundido campos. Revisaría [inaudible], bueno, la referencia del archivo y qué está haciendo el programa antes de modificar algo. Y si el archivo necesita una transformación, la documentaría y conservaría el original.

Emilio: Algo que salió con [Directora ADIP] e [Analista territorial] es que se sepa de dónde viene la información y de cuándo es. Desde la programación, ¿cómo harías para que eso sí esté disponible?

[Desarrollador]: Lo consideraría parte de lo que se carga con la capa, no un texto que agregas al final porque sobró espacio. Que cada conjunto de datos tenga su fuente, la versión o fecha que les hayan proporcionado y una explicación de lo que contiene. Si no les entregaron alguna fecha, no pondría la del día para completar el campo. Mostraría que ese dato está pendiente de confirmar.

Emilio: [Usuaria administrativa] nos decía que una fecha sola puede confundir, porque no sabe si es cuando subieron el archivo o cuando revisaron la información. ¿Cómo lo resolverías?

[Desarrollador]: Tiene razón. Pondría el nombre de la fecha, por ejemplo “fecha de carga” o “fecha de la información”, según lo que realmente represente. Que la capa esté actualisada en la pantalla porque acabas de subirla no significa que describa lo que ocurre hoy. Si mezclas esas cosas, el usuario puede confiar de más. Ese texto hay que acordarlo con quien entrega el dato, no sólo decidirlo desde el diseño.

Emilio: Y si no se actualiza cada año, sino cuando la dependencia hace cambios, ¿cómo pensarías ese proceso?

[Desarrollador]: Lo relacionaría con la entrega de una versión nueva. Tendría que saberse quién la proporciona, quién confirma que se puede usar y cómo se sustituye la anterior. Para el prototipo del semestre, si van a trabajar con archivos fijos, lo pueden documentar así. No hace falta prometer una actualización automática que todavía no existe. Más adelante pueden diseñar ese proceso con la dependencia y probar qué pasa cuando cambian los datos.

Emilio: Si alguien encuentra un error en una capa, ¿sería conveniente que lo pudiera corregir directamente desde el visor?

[Desarrollador]: Yo preguntaría primero qué tipo de error y qué responsabilidad tiene esa persona. Una cosa es avisar que encontró una inconsistencia y otra modificar la información que todos consultan. Para una primera versión podría ser suficiente indicar a quién reportarla. Si más adelante quieren permitir ediciones, necesitarían definir quién las revisa y cómo quedan registradas. Tener un botón para editar no resuelve esas decisiones.

Emilio: Vamos con el ejercicio de las necesidades de información. Si fueras usuario, ¿qué te serviría ver en el mapa, qué en una tabla y qué necesitarías descargar?

[Desarrollador]: En el mapa querría entender dónde está lo que seleccioné y qué extensión tiene. En una ficha o tabla revisaría su clave, su nombre y los datos que permitan reconocerlo. La descarga me serviría si necesito trabajar con esa información en otro programa. Son consultas distintas. También habría que preguntar si la persona busca un elemento específico o quiere comparar varios; eso cambia lo que necesita tener a la vista.

Emilio: Y para alguien como [Usuaria administrativa], que no conoce los términos técnicos, ¿cómo evitarías que se sienta perdida cuando abre el visor? 4

[Desarrollador]: Empezaría por decir qué puede consultar ahí y poner nombres que le ayuden a reconocer las opciones. Si dice “capas”, agregaría una explicación breve. Cuando seleccione algo, la ficha tendría que mostrar claramente qué está viendo. Los detalles técnicos pueden estar disponibles para quien los necesita, pero no tienen que ocupar toda la pantalla desde el inicio. Eso lo probaría con ella, porque algo que a mí me parece obvio puede no serlo.

Emilio: ¿Te serviría que los sectores y los cuadrantes tengan colores diferentes, o con eso no alcanza?

[Desarrollador]: Ayuda, pero también necesitas indicar cuál está activo y qué representa cada color. Si la persona ve dos divisiones encima, puede no distinguir a cuál pertenece la selección. Mantendría un nombre visible y una leyenda sencilla. Además, no dependería solamente del color, porque no todos lo distinguen igual. Una etiqueta o un borde claro puede ayudar. Antes de agregar más cosas, vería si se entiende con una capa a la vez.

Emilio: Con las otras entrevistas salió mucho buscar una dirección. ¿Eso significaría que tenemos que poner un buscador de direcciones desde la primera versión?

[Desarrollador]: No necesariamente. Primero preguntaría qué consulta les pidió la ADIP para esta etapa. Si la tarea es encontrar un sector por nombre o clave, ese buscador puede ser suficiente para empezar. Buscar direcciones agrega otras preguntas: qué servicio las interpreta, qué tan exacto es el resultado y qué pasa si no encuentra una. Puede ser útil, pero conviene que tengan una razón clara para incluirlo y que no lo prometan sólo porque existe en otros mapas.

Emilio: Y si más adelante buscamos una dirección para ver en qué cuadrante está, ¿qué necesitaríamos explicar cuando la ubicación sea aproximada?

[Desarrollador]: Que el resultado es aproximado y que no se puede tratar como una confirmación exacta. Si el punto queda cerca de un límite, esa duda importa. Yo mostraría la información que tienen y dejaría claro qué falta por confirmar. Tampoco concluiría por ese resultado qué autoridad debe atender un asunto. El visor puede mostrar una división, pero esas decisiones dependen de reglas y de personas que conocen el tema.

Emilio: ¿Qué otros errores de la pantalla hacen que una persona piense que la información está mal aunque el problema sea de la herramienta?

[Desarrollador]: Por ejemplo, que busque algo y el mapa se quede en blanco sin explicación. Puede pensar que no existe, cuando quizá no terminó la carga. También que cambie un filtro y la lista no se actualice, o que la ficha muestre el nombre de una selección anterior. Son detalles que afectan mucho la confianza. Hay que revisar que el mapa, la lista y la información del elemento estén respondiendo a la misma consulta.

Emilio: Si se pierde la conexión mientras lo usa, ¿qué sería mejor que apareciera? Te lo pregunto porque a veces sólo sale “error” y ya.

[Desarrollador]: Un mensaje que diga qué no se pudo hacer y qué puede intentar. Por ejemplo, que no se pudo cargar la capa y que puede volver a intentarlo. También distinguiría una coneción interrumpida de una búsqueda sin resultados. No hace falta mostrarle al usuario todo el error técnico. Para quien desarrolla sí puede quedar un registro más detallado. Y si sigues mostrando información que ya estaba cargada, tendría que quedar claro que la nueva consulta no terminó.

Emilio: Tenemos 72 sectores y 1,022 cuadrantes, según lo que nos han explicado. ¿Con eso ya podríamos saber si la página va a ser rápida?

[Desarrollador]: Sólo con la cantidad no. También importa cómo vienen dibujados los polígonos, qué información cargas y qué hace la página cuando te mueves por el mapa. Yo probaría con los archivos que 5

realmente les compartan y en una computadora normal. Revisaría si están volviendo a cargar algo sin necesidad. No prometería que será rápido por tener pocos elementos ni diría que necesita algo muy grande sin probarlo primero.

Emilio: ¿Cómo se relaciona eso con el trabajo remoto? ¿Si a ti te funciona bien ya se puede dar por hecho que a los demás también?

[Desarrollador]: No. Mi computadora y mi conexión pueden ser diferentes. Por eso sirve probar en las condiciones de las personas que van a consultarlo. También dejar por escrito cómo corriste la prueba, porque si otra persona la repite de otra forma, parece que están hablando del mismo resultado y no es así. Remoto, compartir esos pasos te ahorra una llamada larga para descubrir qué cambió entre una prueba y otra.

Emilio: Otra cosa que nos mencionaron son los roles de usuario. ¿Cómo lo entenderías para una herramienta donde algunas personas consultan y otras quizá pueden descargar?

[Desarrollador]: Primero haría una lista corta de las acciones y de quién puede hacerlas. Consultar, descargar y administrar información no son automáticamente el mismo permiso. Después el programa tendría que comprobar esos permisos donde se entrega la información. Ocultar un botón puede ayudar a la interfaz, pero no basta para impedir una acción. Eso sí, tendrían que confirmar con la ADIP qué roles necesitan; no inventarlos porque suenen bien.

Emilio: Y para el usuario, ¿cómo evitarías que piense que algo está descompuesto cuando lo que pasa es que no tiene permiso?

[Desarrollador]: Que el mensaje lo explique. Si la descarga no está disponible para su cuenta, decirlo de una forma clara y señalar cómo consultar esa duda, si existe ese procedimiento. Si no inició sesión, explicar que debe hacerlo. Son situaciones distintas a que el archivo no esté disponible. También evitaría que la pantalla parezca que está cargando para siempre cuando ya sabe que esa acción no se puede completar.

Emilio: Ahorita la idea es un prototipo de uso interno, con las capas que nos entreguen, no una página pública funcionando para toda la ciudad. ¿Eso cambiaría lo que tenemos que hacer?

[Desarrollador]: Sí, cambia el alcance y con quién lo pruebas. Podrían usar un entorno de prueba y concentrarse en el recorrido que necesitan demostrar. Pero sí cuidaría desde el inicio dónde están los archivos y quién entra. No asumiría que todo lo que reciben se puede publicar porque aparece en un mapa. Y dejaría claras las condiciones del prototipo para que una prueba no se confunda con una herramienta que ya está lista para operar.

Emilio: En la parte de descargar, ¿qué revisarías para que no pase que veo una cosa y me llevo otra en el archivo?

[Desarrollador]: Definiría qué descarga ese botón. Si descarga toda la capa, que lo diga. Si descarga sólo los resultados de un filtro, comprobaría que correspondan a lo que seleccionaste. También que el archivo conserve las claves y que se pueda reconocer su versión. Una captura del mapa y un archivo para trabajar con la información cumplen cosas distintas. Yo probaría [inaudible] el archivo después de descargarlo, no daría por bueno sólo que aparezca el botón.

Emilio: Entonces no bastaría con que sí baje algo. ¿Tendríamos que abrirlo otra vez y comparar?

[Desarrollador]: Sí, aunque sea con unos ejemplos conocidos. Ver si está la clave que esperabas, si el nombre corresponde y si el contenido es el que dice la herramienta. Para la versión inicial pueden acordar qué ejemplos usarán con quien conoce las capas. Esa revisíon conecta lo que ve el usuario con el dato que realmente se lleva. Si hay filtros, también probaría qué pasa cuando no hay resultados. 6

Emilio: ¿Cómo harías las pruebas con usuarios? Porque nos han dicho que no sólo preguntemos si les gusta la página.

[Desarrollador]: Les daría una tarea concreta, como encontrar un cuadrante por su clave y decir qué fuente aparece. Los dejaría intentar sin señalarles todos los botones. Después preguntaría qué pensaron que significaba cada opción o dónde dudaron. Si les explicas el recorrido antes, pruebas que siguieron tus instrucciones. También observaría si creen que terminaron cuando en realidad están viendo otro elemento. Eso puede pasar aunque digan que la página les gustó.

Emilio: Y si alguien no termina la tarea, ¿cómo sabemos qué tenemos que cambiar? ¿Se arregla haciendo el botón más grande?

[Desarrollador]: A veces, pero no siempre. Puede no haber encontrado el botón, no entender el nombre o no reconocer el resultado. Yo preguntaría qué esperaba encontrar justo en ese momento. También anotaría los pasos, porque un error al principio puede explicar lo que pasó al final. Antes de cambiar toda la pantalla, intentaría identificar cuál de esas cosas falló y probar una modificación que responda a esa duda.

Emilio: ¿Qué cosas de accesibilidad revisarías desde una primera versión? Algo que podamos tener presente aunque todavía no tengamos una herramienta enorme.

[Desarrollador]: Que los textos se lean, que puedas distinguir las opciones y que los botones tengan nombres comprensibles. También probaría navegar con el teclado y aumentar el tamaño de la página. Si una ficha sólo se abre pasando el ratón, puede dejar fuera a alguien. Y, como te decía, que el color no sea la única manera de entender la información. Son cosas que pueden revisar temprano, no esperar hasta que esté terminada toda la herramienta.

Emilio: Si somos varias personas trabajando en el proyecto, ¿cómo evitarías que cada quien tenga una versión distinta y después no sepamos cuál entregar?

[Desarrollador]: Usaría un lugar compartido para el código y una forma clara de identificar los archivos de datos. Lo importante es poder saber qué cambio hicimos y qué versión estamos probando. Un mensaje de “te mandé el último” se vuelve confuso muy rápido. Tambien anotaría las decisiones pequeñas que afectan a todos, como qué campo se usa para buscar. Si cambias eso, otra persona tiene que enterarse antes de probar algo diferente.

Emilio: ¿Y si un cambio nuevo rompe algo que ya funcionaba? Porque me imagino que tampoco todo sale bien a la primera.

[Desarrollador]: Claro, pasa. Primero necesitas poder reconocer qué cambió y volver a una versión que funcionaba mientras lo revisas. Eso aplica al programa y a los archivos. Si cambias los dos al mismo tiempo, cuesta más encontrar la causa. Yo separaría los cambios cuando se pueda y repetiría las consultas importantes. No tiene que ser una prueba enorme, pero sí revisar que buscar, seleccionar y descargar sigan haciendo lo que acordaron.

Emilio: Si tuvieras que ayudarnos a escoger por dónde empezar, ¿cuál sería el primer recorrido que desarrollarías?

[Desarrollador]: Con lo que me has contado, empezaría por abrir una de las capas, encontrar un elemento y consultar su información de origen. Luego comprobaría la descarga, si forma parte de lo que les pidieron. Cuando eso funciona completo, ya puedes mostrarlo y escuchar dudas reales. Si empiezan por hacer todos los menús, pueden tener muchas pantallas y todavía no poder responder una consulta sencilla.

Emilio: Y desde tu experiencia, ¿qué sería lo que nos podría hacer perder más tiempo en este semestre? 7

[Desarrollador]: Meter funciones sin una pregunta detrás. Por ejemplo, querer agregar rutas o muchas estadísticas porque se ven bien en otros mapas. También elegir una solución que nadie del equipo entiende y luego depender de una sola persona para cambiarla. Yo buscaría que puedan explicar cómo funciona lo que están construyendo y que otra persona pueda retomarlo. El tamaño del proyecto tiene que corresponder al tiempo que tienen y a lo que van a probar.

Emilio: ¿Hay algo de lo que te he contado que tú necesitarías preguntar antes de empezar? Algo que todavía no te quede claro del encargo.

[Desarrollador]: Sí. Quiénes van a hacer las primeras consultas, qué necesitan encontrar y qué archivos van a recibir exactamente. También cómo se espera acceder al prototipo y qué consideran una entrega satisfactoria. Y preguntaría quién puede confirmar los resultados de las pruebas. De la operación de la ADIP no te podría responder por mi cuenta; eso sí tendrían que hablarlo con [Directora ADIP] y con las personas que ella les indique.

Emilio: Ya para cerrar, ¿cómo sabrías que esa primera versión sí está cumpliendo, más allá de que el mapa se vea bonito?

[Desarrollador]: Que una persona pueda hacer la consulta que acordaron, reconocer el resultado y entender de dónde viene. Que ustedes puedan comprobar que el programa está mostrando el archivo correcto y explicar qué pasa si algo falla. Si hay una descarga, que corresponda a lo que promete. Y que las dudas que salgan en las pruebas queden registradas. No diría que todo está resuelto; diría qué recorrido pudieron comprobar y qué sigue pendiente.

Emilio: Va. ¿Algo más que creas importante y que no te haya preguntado?

[Desarrollador]: Mmm, quizá que no conviertan todo en una decisión técnica. A veces el problema es que nadie sabe quién confirma una fecha o quién entrega la siguiente versión. Puedes programar más botones y esa duda sigue ahí. Y desde el trabajo remoto, dejaría [inaudible], dejaría una explicación corta de las decisiones, porque quien retome el proyecto no va a haber escuchado todas estas conversaciones.

Emilio: Perfecto. Para ver si entendí bien: desde tu parte, lo principal sería definir la consulta antes de programarla, distinguir los errores de los datos de los de la herramienta y dejar claro quién confirma cada cosa. También probar el recorrido completo y que lo que se ve corresponda con lo que se descarga. ¿Sí va por ahí?

[Desarrollador]: Sí, así es. Y empezar con algo que puedan comprobar. Después pueden agregar más funciones, pero que cada una tenga un uso claro. Yo les puedo ayudar a revisar cómo se comporta el programa; para confirmar que la información representa bien el territorio necesitan a las personas que la conocen. Si esas dos partes se hablan desde el principio, se ahorran varias vueltas.

Emilio: Va, [Desarrollador], muchísimas gracias por tu tiempo. Nos sirve mucho tener esta perspectiva porque no queremos quedarnos sólo con la parte de cómo se ven los mapas. Vamos a ordenar lo que platicamos y llevar esas dudas con [Directora ADIP] para decidir qué podemos probar en esta primera versión.

[Desarrollador]: Claro, Emi, con mucho gusto. Cuando tengan algo para probar, ya pueden revisar una consulta paso por paso y ver dónde se atora. Ojalá les sirva y les vaya bien con el proyecto. 8